Worldwide legislative electoral systems visualisations using dimensionality reduction & clustering

github.com/yaya7211 github.com/Romain-Jaffuel

Abstract

Legislative elections happen all around the world, processed using different electoral systems. Although their results can't be compared directly, their scores, if processed, are the perfect fit for data visualisations. We can clearly witness electoral system clusters across similar countries, differences among them, and finally changes within the same countries. The results were then framed using literature to reveal the mechanics behind the electoral system. The case study of the Russian Federation reveals both the diachronic shift in the voting behaviour between two elections and the lack of data for specific countries in the dataset.

Authors: BENAMAR Yassine, JAFFUEL Romain **Date:** November 2025

Table of contents

1. Introduction

1.1. Background & Context

The emergence of data science strengthened the possibility for researchers to understand the dynamics working in the background of modern democracies. A lot of studies are conducted on the national results of every election, trying to better frame the behaviour of the citizens through data analysis. Not every study uses a constituency level to analyse the mechanics, while a lot of social science researchers proved the relevance of studying the local level when conducting work on electoral dynamics. This is what led us to explore the differentiated behaviour of regions and countries through data that includes the local level.

Dimensional reduction methods and clustering algorithms are highly effective in our type of research. Indeed, we have a really high amount of data as we cover almost 150 years of elections in the entire world, each of them having local quantitative results for each political party, turnout scores, and more. Both to analyse visually similarities or apply a clustering algorithm, it is easier to first reduce the dimensionality.

1.2. Relevant Literature

In 2020, Moses & Box-Steffensmeier, two researchers at the Ohio State University, wrote a guideline on how to use machine learning on election datasets. In the abstract of their paper, they enlighten the relevance of those kinds of approach: “Improving the accuracy of outcomes, refining measurements of complex processes, addressing non-linearities in data and introducing new kinds of data may be achieved using machine learning.”. They then describe methods specifically for unsupervised learning, which we partially used in our study.

Arend Lijphart is recognised as one of the specialists on electoral systems. In “Patterns of democracy” (2012), he states: “clear patterns and regularities appear when these institutions are examined from the perspective of how majoritarian or how consensual their rules and practices are”. This statement well describes why we think our study is relevant to solidify the analysis previously conducted.

On the website where the data are available, they describe the project with the following statement: “The Elections Archive (THEA) is a repository of subnational election results and associated materials. Our motivation is to preserve and consolidate these valuable data in one comprehensive and reliable resource that is ready for analysis and publicly available at no cost. This public good is expected to be of use to a range of audiences for research, education, and policy-making.”

1.3. Research Aims & Objectives

The main objective of this study is to explore and cluster legislative electoral results across time and countries using constituency-level data. Using quantitative indicators, the analysis aims to identify recurring electoral patterns that appear when elections are compared beyond a national perspective.

A second objective of this work is to assess the role played by electoral systems in shaping these patterns. By examining whether the electoral system tends to be associated with specific structures in the data. The study seeks to better understand how institutional rules influence the organisation of electoral results observed in the clustering.

2. Methods & Tools

2.1. Dataset Description

As we aimed to compare elections on a worldwide scale, we had to choose the election that is the most comparable across most countries; Lower Chamber legislative elections. In comparison, Presidential elections aren't held in a large enough number of countries (whose State chief is appointed out of the legislative majority), and Higher Chamber legislative elections/appointments aren't as relevant as Lower Chamber for turnover data.

Being aware that countries don't share the same electoral system, we decided that we should only take into account cumulative amounts of votes per party, per country, per election, such that differences between proportional and plurality systems won't interfere with the analysis.

The perfect fit for these specifications is the Constituency-Level Election Archive Lower Chamber Election Archive of 2025 (CLEA - 2025).

Every year, The Election Archive (THEA) summarizes in large datasets a wide amount of elections in every country from 1870 to 2025. Every update of the dataset is just the addition of the past document with the newest election results.

The dataset comes in xlsx format, split in two parts, alphabetically ordered, from A to J included then from K to Z. It comes with a few PDF files, some of which directly linked to the metadata of the datasets, and some of which literature on the election systems country by country, specifying which election is represented in the dataset and which isn't.

The datasets share the same columns, first two identify the election by an ID and its release in the CLEA datasets.
The second three columns are geography-related, specifying the continent and the country. Each country has a code that identifies it in case of a regime or name change. For example, France keeps the same country code for its 1910 legislative election (IIIrd Republic) and 1973 (Vth Republic).
The two following columns are date-related: year and month of the election.
The four following columns are constituency identifiers, by name, code, and sub-national region.
The next ten columns are the most interesting ones, as they are the ones at the core of our analysis. They concern the party running for election during the first round, the number of eligible voters, the turnout, the total number of votes, the number of valid votes, the votes the party/candidate got, its share, and its party's cumulative votes if it is a nationwide party.
The rest of the columns are related to the second round, if there is one. Most countries don’t hold them, so these columns will quickly be dropped.
The last column is the number of seats won by that party at that election.

2.2. Preprocessing & Cleaning

The first step was to transform the dataset to CSV format, as XLSX was way heavier to process every time, then to concatenate it into one single dataset.

The second step was to look for missing data. But this was a trickier part than we initially thought.
At first glance, it seems like the dataset is almost full, as we can see in the following MissingNo matrix.

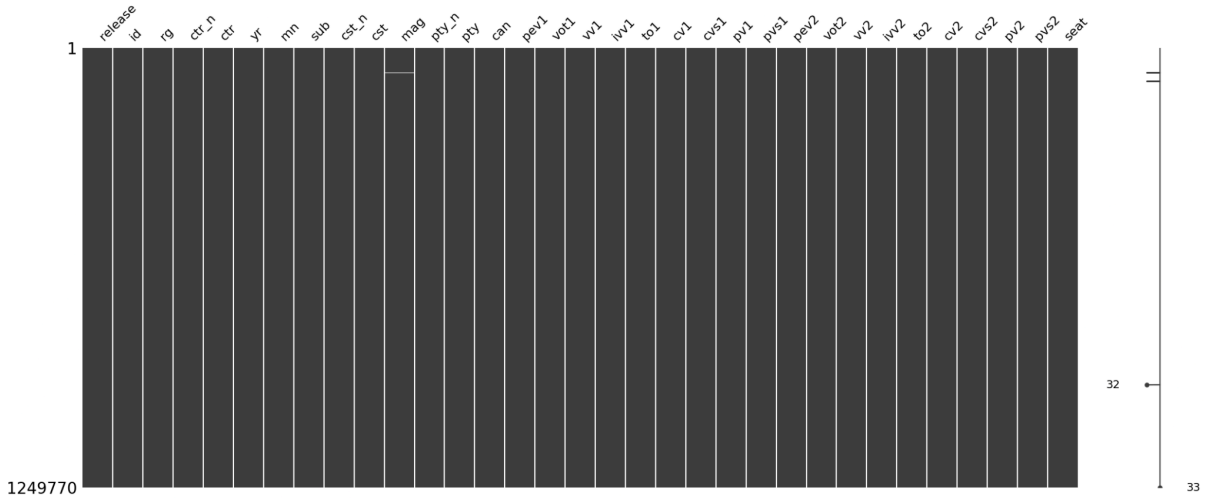

Fig 2.2.1 : MissingNo matrix for the concatenated Dataset

However, this dataset doesn't handle missing data with NAs, but rather with custom codes, they give insights on the reason the data is missing. This information is written in the "Codebook" PDF that comes along with the dataset.
Missing data are split into three categories ;

- Missing Data, represented by -990
- Uncontested elections (Single candidate), represented by -992
- Suspended elections, represented by -994

It is therefore relevant to take into account these special values to plot the missing data matrix. We did it by reimplementing the missingno tool, tuning it to plot in different colors all of these 3 special values, and NAs, which gave us the following plot.

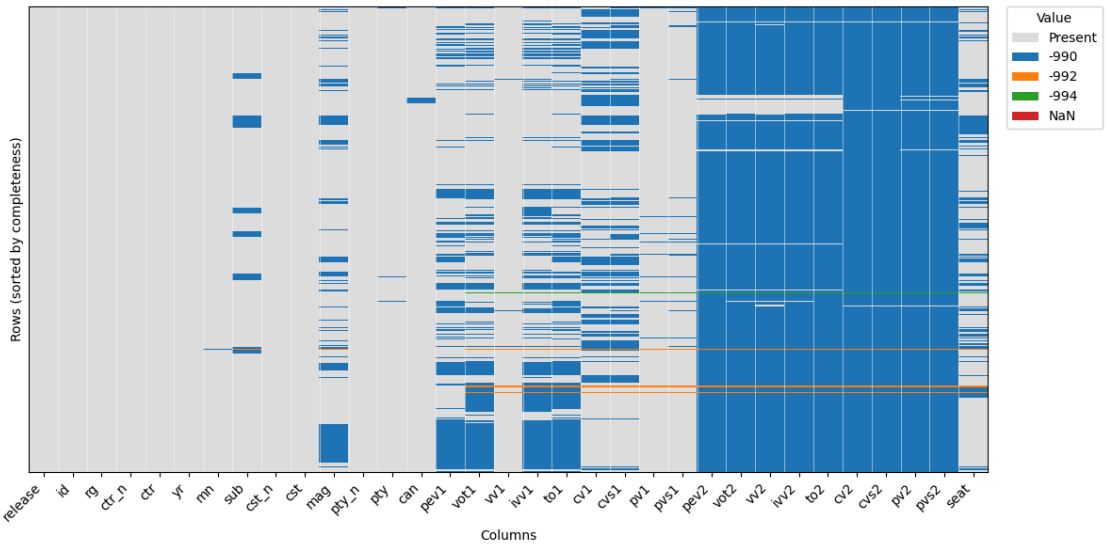

Fig 2.2.2 : MissingNo with custom NA values

As we can see, most of the first columns are almost full, and most of the last ones (except the very last one), are almost only missing data.
This can be easily explained by the fact that these columns are the second rounds of the elections, which often doesn't exist in most election systems. This is the reason why these columns will be dropped in our analysis. Our focus will be on the first round.

Our third step was to check which variables are suitable for a quantitative analysis using unsupervised models.
We first checked for correlations between variables. The following heatmap is the correlation matrix between variables.

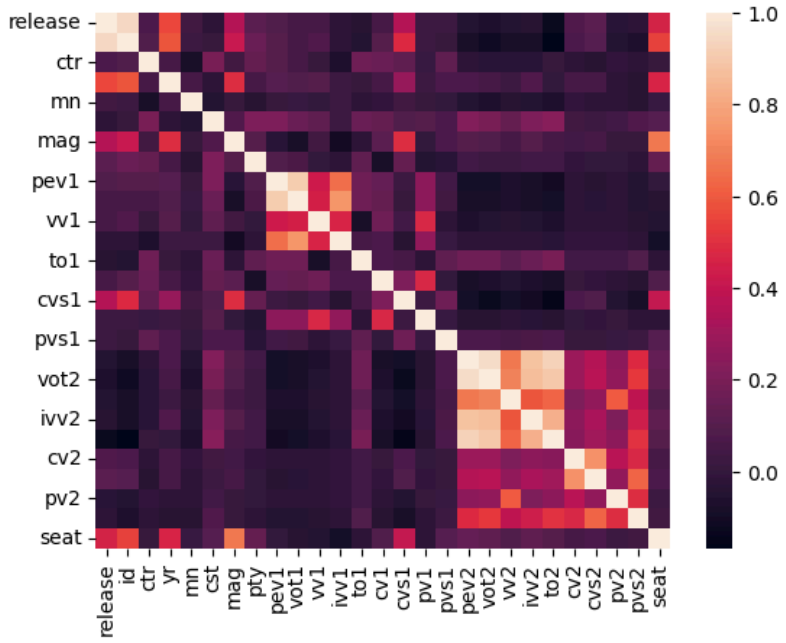

Fig 2.2.3 : Correlation Heatmap

As we can observe, the only obvious correlations are between the columns that describe the same round of elections. For example, pev2, vot2, …, pvs2 are highly correlated since they all contain high amounts of -990 for elections that are only one-rounded. This is one more reason to drop them.

Finally, focusing on quantitative variables of the first-round, we checked their distribution to select only relevant algorithms. The focus is on the following 9 variables ;

- pev1 : Number of eligible voters
- vot1 : Casted votes
- vv1 : Valide votes
- ivv1 : Invalid votes
- to1 : Turnout
- cv1 : Candidate votes
- cvs1 : Candidate vote share
- pv1 : Party votes
- pvs1 : Party vote share

The distribution of these unprocessed variables are on the following plot ;

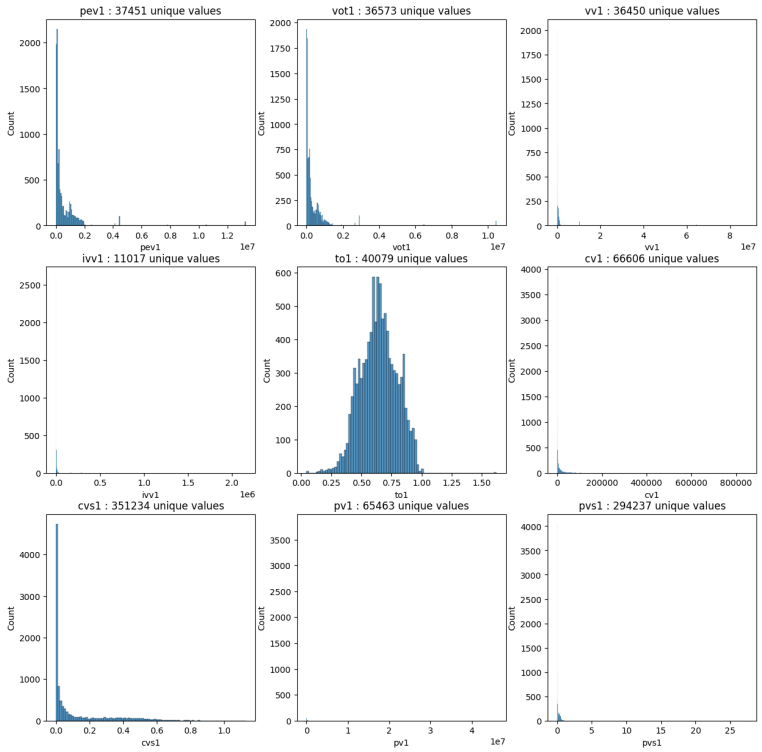

Fig 2.2.4 : Distribution of the raw 9 selected variables

As we can see, most of them are VERY 0 inflated, such that most models will be unsuitable for any relevant analysis that relies on normality.

Our first attempt to tackle this issue is transforming the data.
We first tried the classical log(x+1). It worked surprisingly well. The result is in the following plot ;

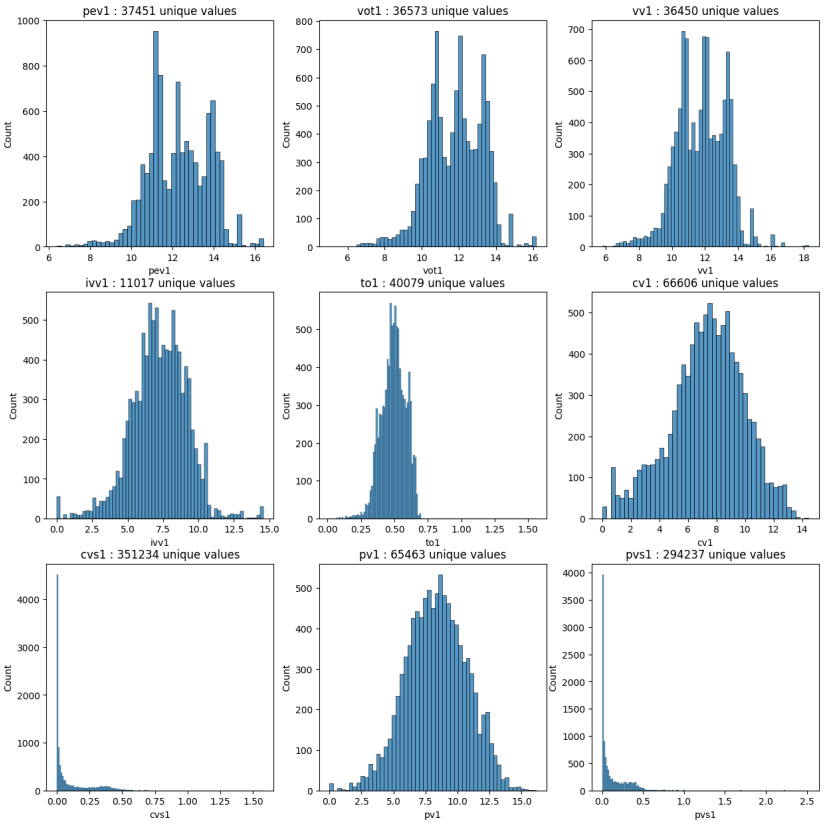

Fig 2.2.5 : Distribution of processed 9 selected variables

Most of the variables looked more centered around a mean value, most of the tails are symmetrical. Yet most of them are not perfectly normal, and two of them are still 0 inflated poisson shaped.

We finally standardized our data using the standard scaling formula, using both means and variances.

To analyse the type of electoral system in each country at each election, we looked at the “mag” column, corresponding to the magnitude of the district. It corresponds to the number of seats allocated in the legislative assembly to each territory. For each election, we counted how many constituencies exist in total, how many have a single-member district (mag = 1), and how many have multi-member districts (mag > 1). Based on these counts, we added the result in an election type column with “Single-Member Plurality” (SMP) when all districts are single-member, “Proportional Representation” (PR) when all districts are multi-member, and “Mixed” when both types coexist; elections that do not match any of these patterns were labelled as Unknown.

2.3. Theoretical Concepts

Before going to clustering models, we first ran our variables through some dimensionality reduction algorithms, as we still have too much data for any meaningful data visualisation, and to have a better understanding of explained variances.

The first dimensionality reduction algorithm we used is Principal Component Analysis. Ran on transformed and standardized data, we got the following explained variances ;

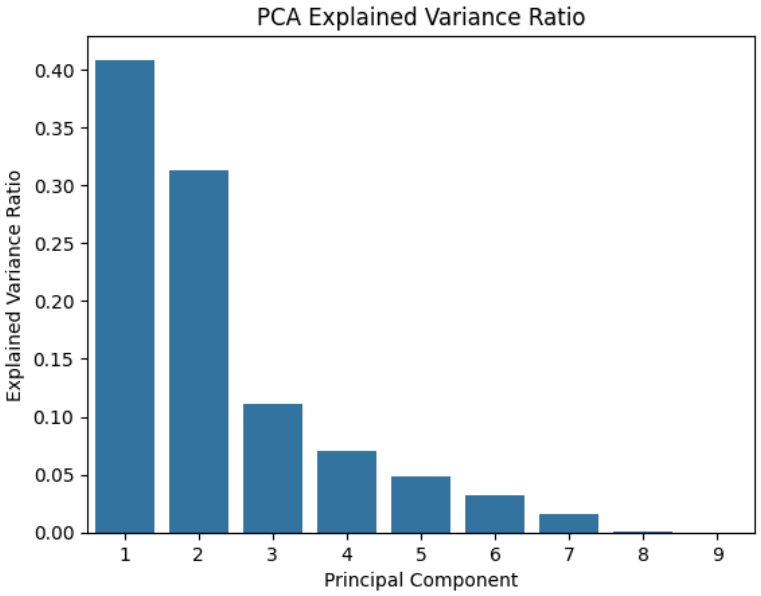

Fig 2.3.1 : PCA explained variances

Pros of PCA is that this model preserves as much important data as needed. Dimensionality reduction doesn't lose any relevant data for the following unsupervised models. For the following data visualisations, we will focus on the first 3 PCAs, as they explain around 80% of the data.

The clustering algorithms we tried are the Kmeans clustering and the DBSCAN.
The first one, KMeans, simply looks for a set of centroids around which the closest data points belong to the same cluster.
The second one, DBSCAN, looks for datapoints in the most densely populated areas, these datapoints are considered core points of the cluster. From these datapoint, neighboring ones are either core points, or border points if they don't have enough neighbors.

t-SNE is a non-linear dimensional reduction method more focused on preserving local neighbourhood structure over the global structure of the dataset. Using conditional probability, t-SNE compares pairwise similarities over each point. The perplexity parameter controls the number of neighbouring points considered for the distribution calculus. We set it at 30 to have a good preservation of the local structure. t-SNE then sets the Kullback-Leibler divergence between the high-dimensional and the low-dimensional similarities, and minimises this divergence using gradient descent. t-SNE is useful to have stable local similarity patterns, but it can't be used to interpret the inter-cluster distance. (Van Der Maaten, 2014)

Here is an overview of the corresponding algorithm:

t-SNE algorithm:
begin

1. Compute pairwise affinities $p_{j|i}$ with perplexity *Perp*
2. Set $p_{ij} = \frac{p_{i|j}+p_{j|i}}{2n}$ (n - Number of data points)
3. Sample initial solution $Y^{(0)} = y_1, y_2, ..., y_n$ from $N(0, 10^{-4}I)$
   for t=1 **to** T **do**
   1. Compute low-dimensional affinities q_{ij}
   2. Compute gradient $\frac{\delta C}{\delta y}$
   3. Set $\mathcal{Y}^{(t)} = \mathcal{Y}^{(t-1)} + \eta\frac{\delta C}{\delta y} + \alpha(t)(\mathcal{Y}^{(t-1)} - \mathcal{Y}^{(t-2)})$

   end

end

On step 2, the Pj|i is computed as follows:

$$p_{j|i} = \frac{\exp\left(\frac{-\|x_i - x_j\|^2}{2{\sigma_i}^2}\right)}{\sum_{k\neq i}\exp\left(\frac{-\|x_i - x_k\|^2}{2{\sigma_i}^2}\right)},$$

As seen above, t-SNE requires a lot of computation. This is why we used a reduced random sample of 20,000 lines, allowing us to compare whether countries produce a good parameter to cluster similar elections.

To study specific countries, we applied a subset of data that uses a disproportionate number of rows. For our case study on the Russian Federation, the subset contained the 4,174 rows about the 2003 and 2016 elections in Russia – the only ones available – and was completed with random data from the whole dataset to achieve a total of 20,000 rows. This gave us an easier visualisation of the difference between the two elections.

Because t-SNE doesn't keep the global structure of the data to focus more on local structure, we then used the UMAP method to solidify our analysis

UMAP is also a non-linear dimensionality reduction method focused on preserving local neighbourhood structure. It also retains more global structure, unlike t-SNE. UMAP first models the high-dimensional dataset as a weighted k-nearest-neighbour graph. The parameter n_neighbors controls the number of the local neighbourhood used to construct the graph. Higher values emphasise more global structure, while lower values focus on local relationships. UMAP then constructs an embedding by minimising the cross-entropy between the high-dimensional and low-dimensional graph representations. Optimisation is performed using stochastic gradient descent. UMAP is well-suited to identify stable neighbourhood patterns and preserve relative global organisation, but distances and cluster densities in the embedding should still be interpreted with caution. (McInnes, 2018). As the computation requirement is lower than for t-SNE, we used a subset with a random sample of 50,000 rows.

2.4. Computational Tools

This analysis has been run through a Python Jupyter Notebook, using popular tools of data processing and numerical computation, such as NumPy and Pandas, as well as tools dedicated to missing data exploration, such as missingno.

Data visualisation relies on widely used Python libraries, such as Matplotlib for static plots, including advanced visual components from matplotlib.colors, matplotlib.patches and matplotlib.lines, Seaborn for statistical visualisation, and Plotly for interactive graphics.

Dimensionality reduction and preprocessing are carried out using Scikit-learn, notably through StandardScaler for feature normalisation, PCA for linear dimensionality reduction, and t-SNE for non-linear embedding. Clustering methods are also implemented using Scikit-learn, including KMeans, DBSCAN, and nearest-neighbour computations via NearestNeighbors, with parameter selection supported by the kneed library. Density-based clustering is further extended using HDBSCAN.

The whole lab is available for download and rerun through the GitHub link in the front page.

3. Results

3.1. Descriptive Statistics

Before digging into inferential statistics, the first step is to take a look at the common descriptive statistics ; quantiles, means, standard deviations etc. These results are as follows.

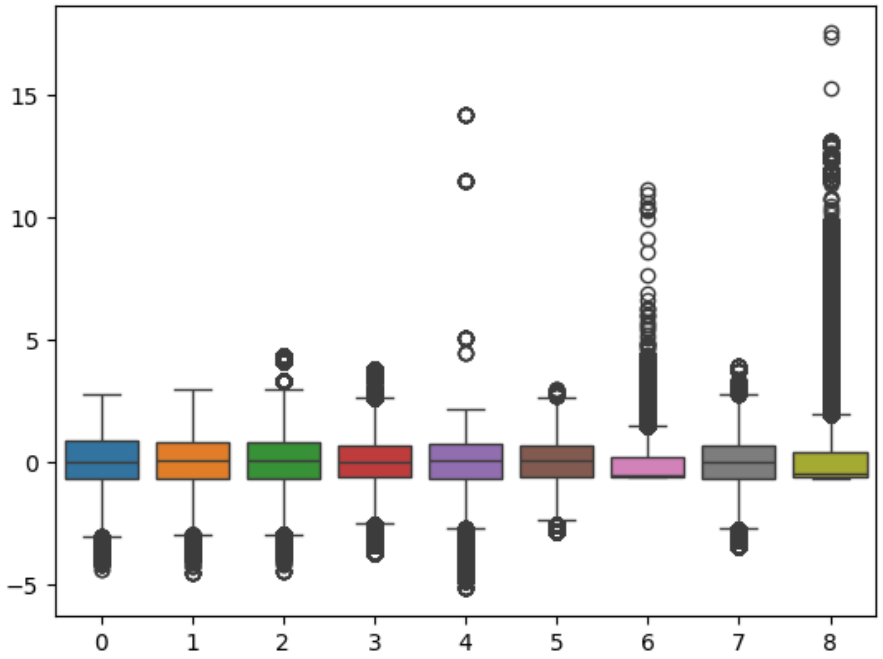

Fig 3.1.1 : Boxplot of transformed & scaled data

Unsurprisingly, all data have the same standard deviation (around 1), the same mean (around 0), and the last two of them are full of outliers. This does make sense considering the distributions we plotted in the previous part. This plot confirmed the scaling and the transformation worked well, we may thus proceed to the clusterings.

Lastly, before running any clustering algorithm we can visualise the PCA transformed data with the first 3 dimensions (that explain 80% of the variance).

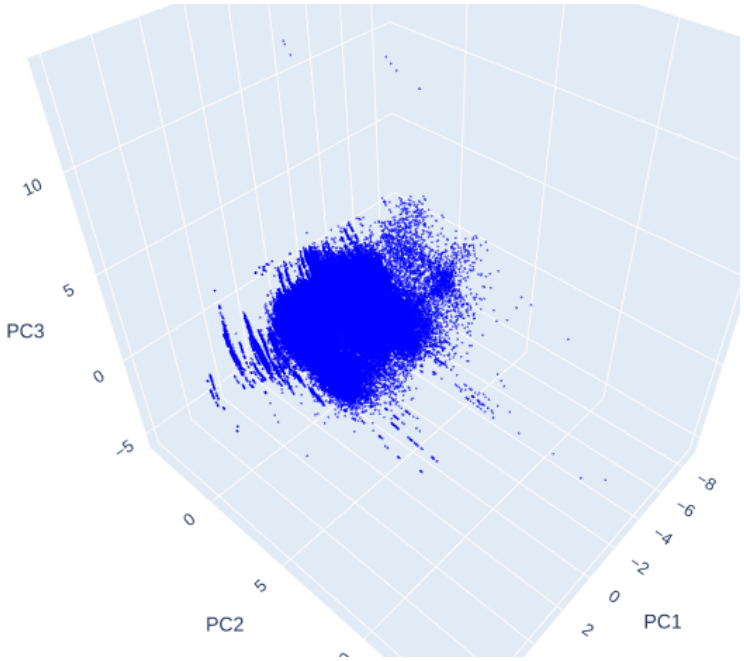

Fig 3.1.2 : 3D Scatter plot of unclustered data

As we can see, most of the data points are clustered in the same cloud. It looks like there are some kind of unconnected layers at the front, and a less dense cloud at the back.

3.2. Clustering Results

The first clustering algorithm we tried was the K-Means clustering.
Before running it, we have to compute the optimal number of clusters to look for, as they aren't obvious by eye. The elbow method gave us the following result ;

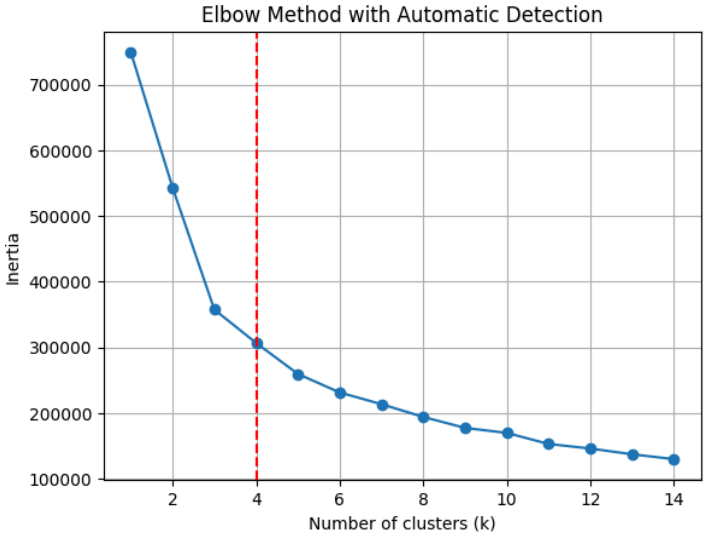

Fig 3.2.1 : Elbow method for KMeans Clustering

As we can see with the red line (automatically computed using KneeLocator), the optimal number of clusters is 4. We can highly doubt this as we can visually not see how can four spherical clusters give any meaningful insight on our data.
Here is the result of the KMeans Clustering ;

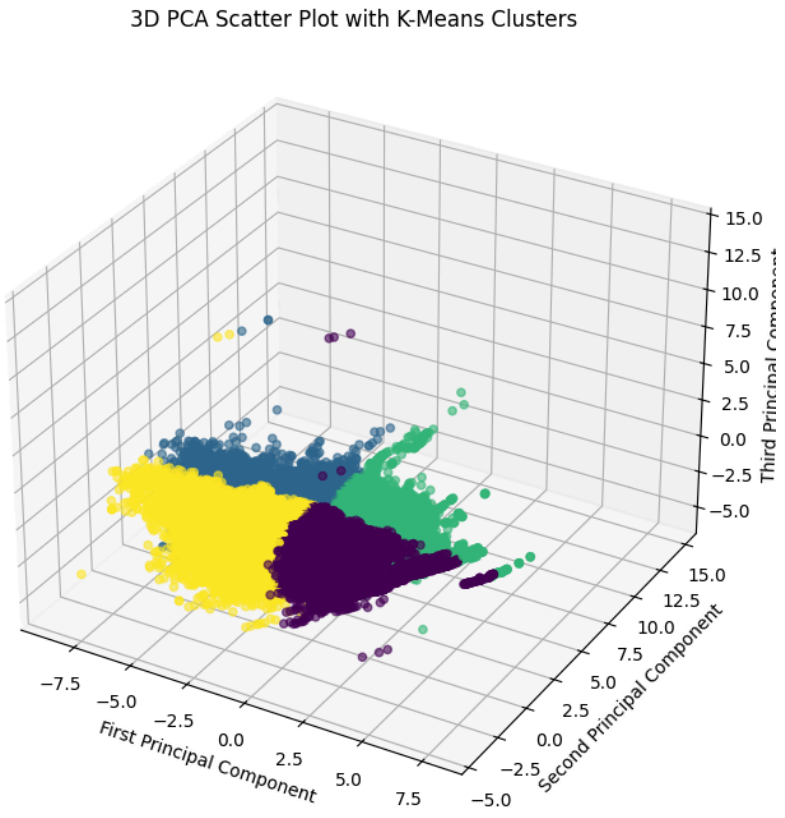

Fig 3.2.2 : Kmeans Clustering

As we expected, the clustering split the datapoint in what seems to be arbitrary lines. We can't interpret anything interesting from this clustering. Our best option is to try another clustering method.

Before running DBSCAN, we need to tune it. We can assert that most data points that are at the core of any cluster have at least 5 neighbors, as the cloud is HEAVILY dense.
However, we still have to decide the epsilon parameter of our clustering. To do so, we use the K-Distance plot, which displays the distribution of distance between points. Here is the result.

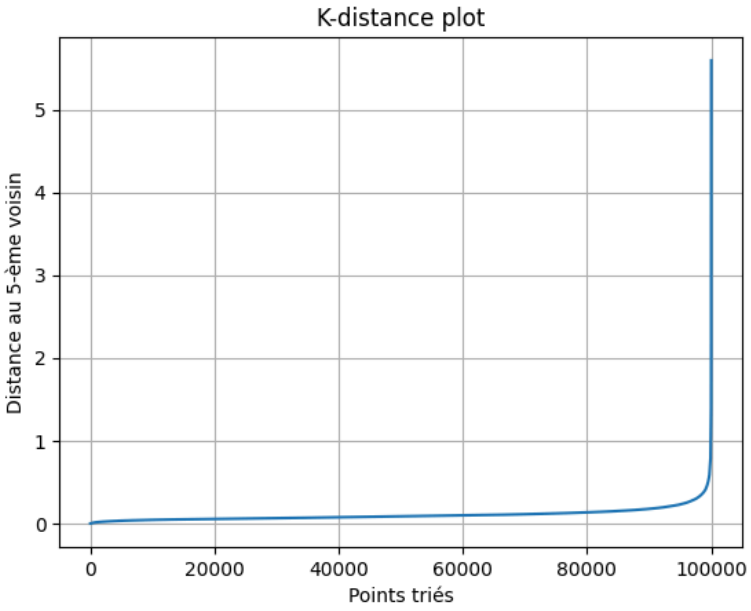

Fig 3.2.3 : K-Distance plot for DBSCAN

As we can observe, the elbow is somewhat around 0.3 and 0.4. We can decide to set the epsilon at 0.35, and the minimum sample size to 5 neighbours. Here is the result of this clustering algorithm.

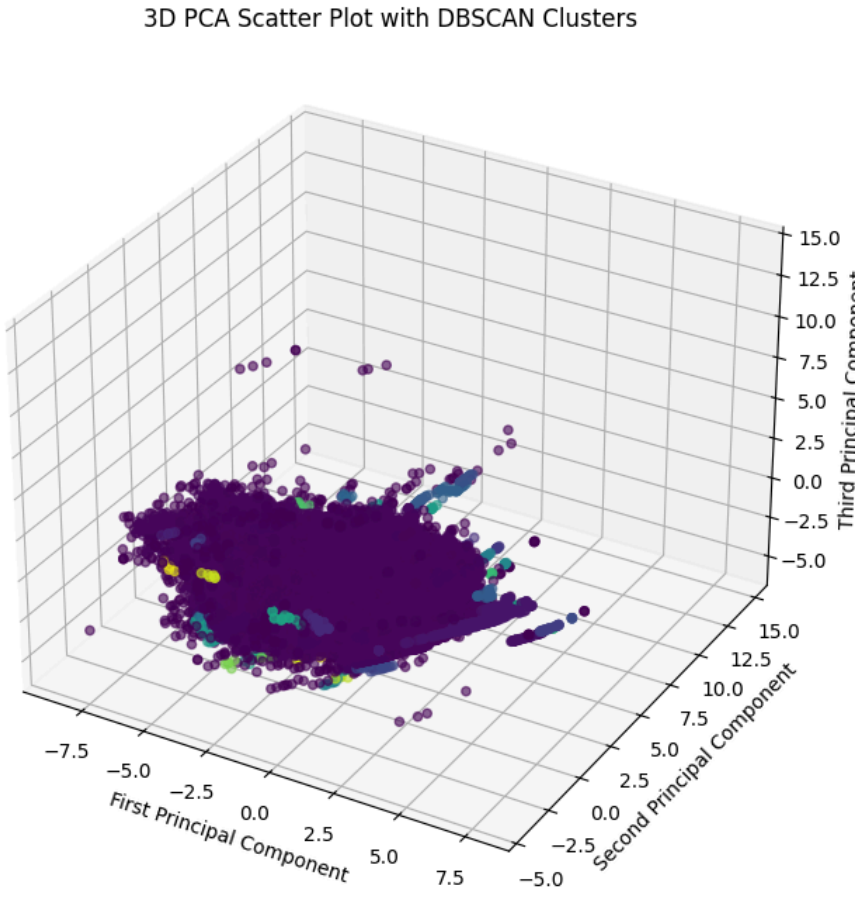

Fig 3.2.4 : DBSCAN

We can see that there is only one meaningful cluster that contains most of the datapoints. Most of the other clusters are a lot, but contain only a few data points a bit further than the main cloud. We can ensure this affirmation with this histplot ;

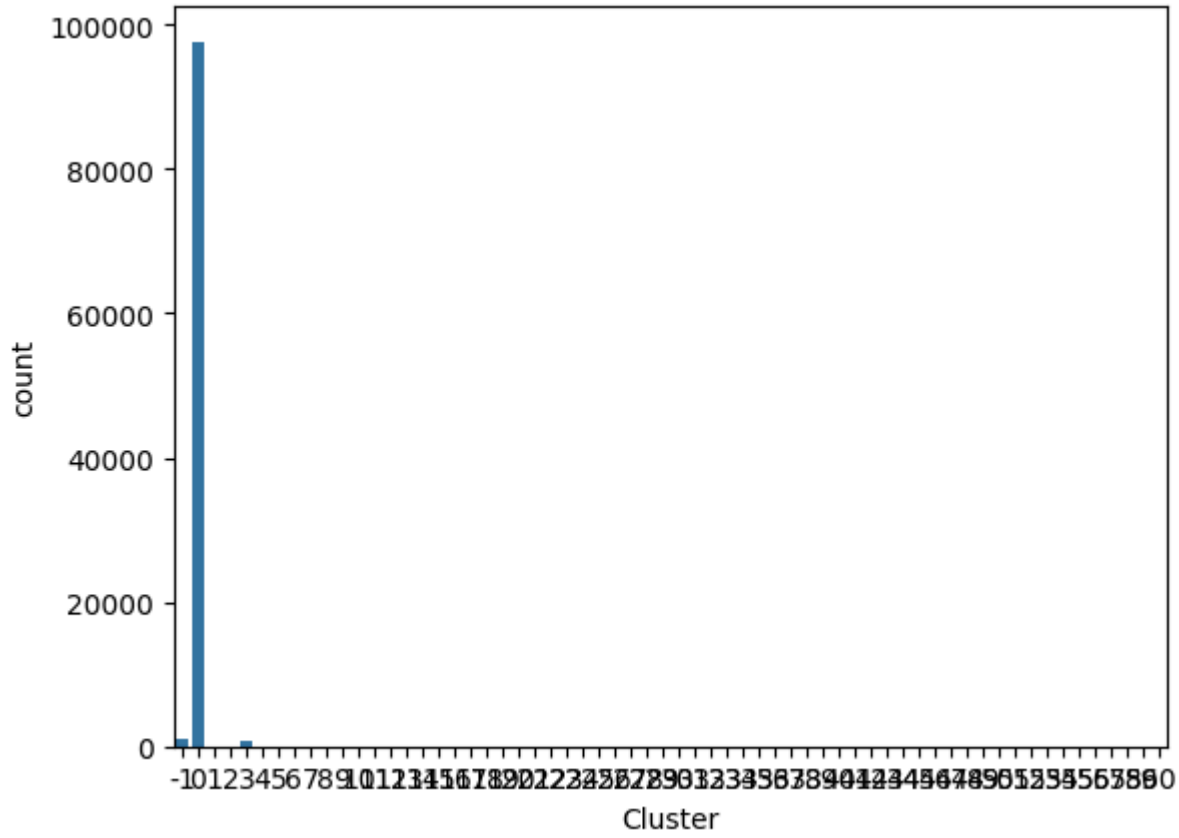

Fig 3.2.5 : Histogram of datapoints per cluster in DBSCAN

Most of datapoints are in the main cloud, a few of them are unlabeled, and there are more than 60 other clusters that contain only a few datapoints. We can't interpret anything meaningful out of this clustering.

One last attempt would be to use a more advanced method of DBSCAN that would be sensitive to region's density, as DBSCAN isn't. We can use HDBSCAN. Here is it result ;

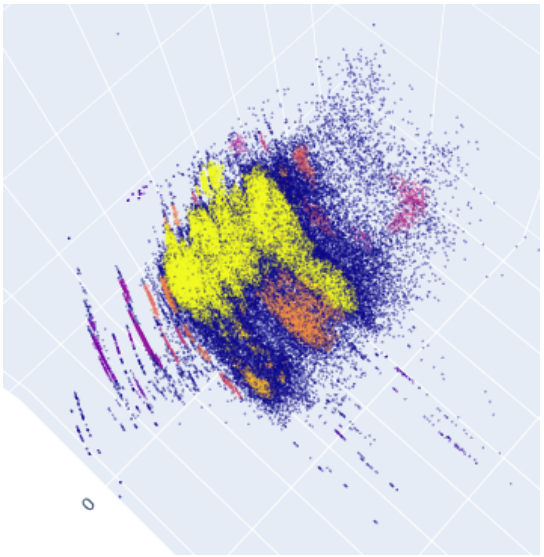

Fig 3.2.6 : HDBSCAN Clustering

As we can see, some new clusters emerge, but barely readable and interpretable. However, these precious informations can be more easily displayed using the following dimensionality reduction and clustering methods.

This figure depicts a DBSCAN clustering performed on the low-dimensional representation of the dataset after applying t-SNE. We can see that some clear clusters are formed. However, t-SNE representation can’t be used to interpret intercluster distance. But we wanted to understand the phenomenon behind those clusters. This is why we then plotted the same t-SNE graph, but colouring the points according to the assigned country.

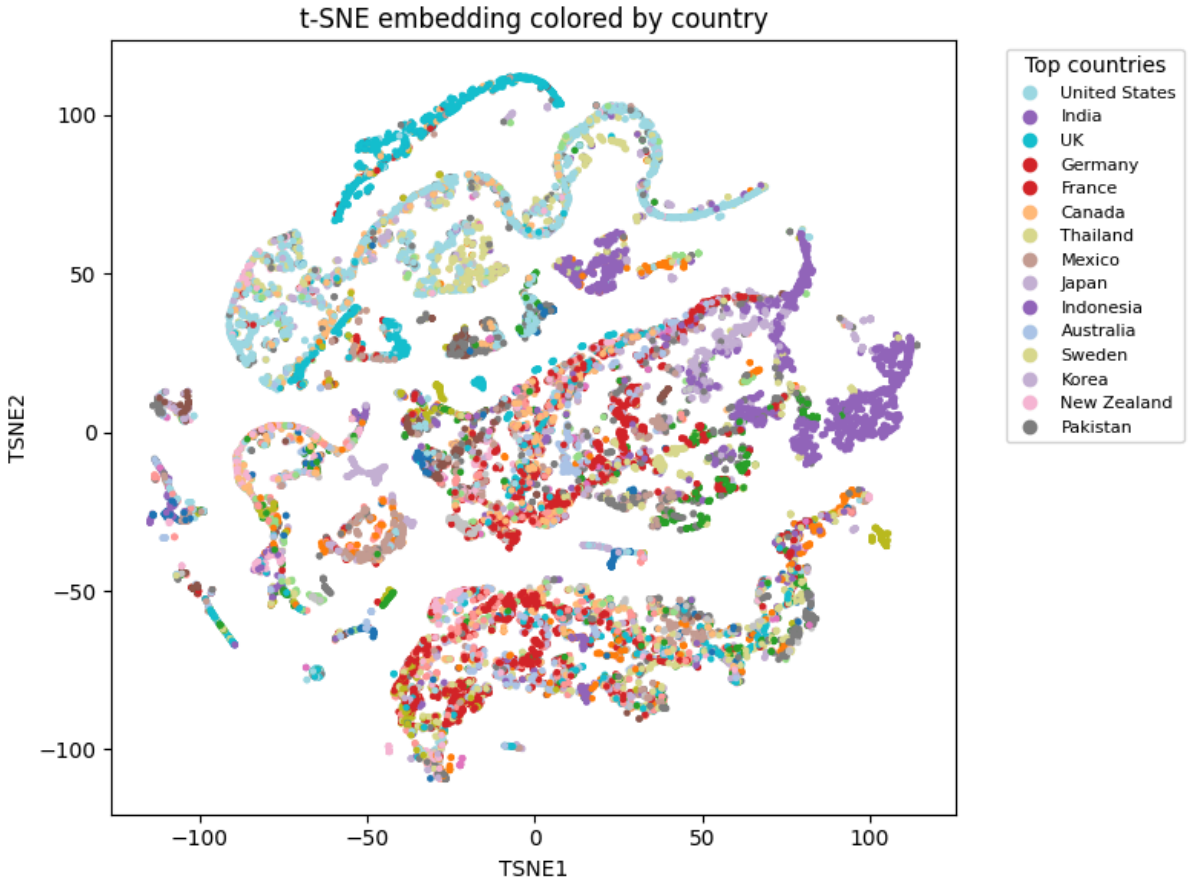

Fig 3.2.7 : t-SNE Clustering per country

This figure shows the countries' distribution after t-SNE dimensional reduction. The legend only includes the fifteen countries with the most data available. We can see that some countries are organised in clusters, like the UK at the top of the figure. Some other clusters mixed a large number of countries, like in the middle of the figure. We also need to take into consideration that the amount of data is disproportionate among the countries, which influences the way the t-SNE

dimensionality reduction is performed. It is interesting to see that both UK and India are countries with specific cluster for them according to the fact that India was a colony of the UK.

We then searched to see how the electoral system was represented in this dimensionally reduced environment.

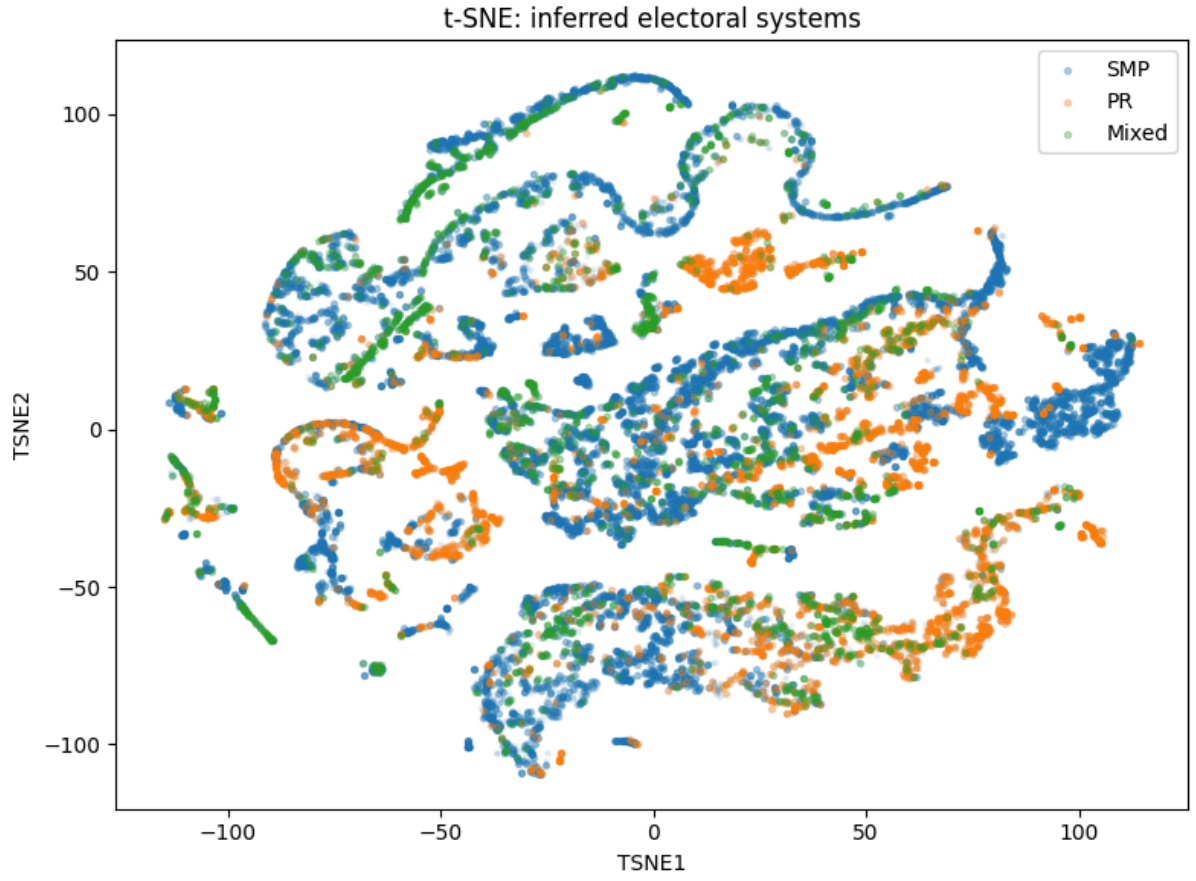

Fig 3.2.8 : t-SNE per electoral systems

This figure shows the distribution of the different electoral systems after the t-SNE dimensional reduction. The three types are Single Member Plurality (SMP), Proportional Representation (PR) and when an election includes both systems (Mixed). We can see that some clusters include multiple system like in the central cluster, while some others consist of one dominant system. The result is coherent because the mixed green dots are the ones closest to both blue and orange dots. Mixed election type doesn't represent a unique cluster anywhere, while we can see a cluster only for SMP or PR.

We then wanted to solidify our study by including a case study of a specific country's behaviour regarding different elections. We chose Russia as it is documented as having many reforms and chronological changes. (Turchenko, 2015)

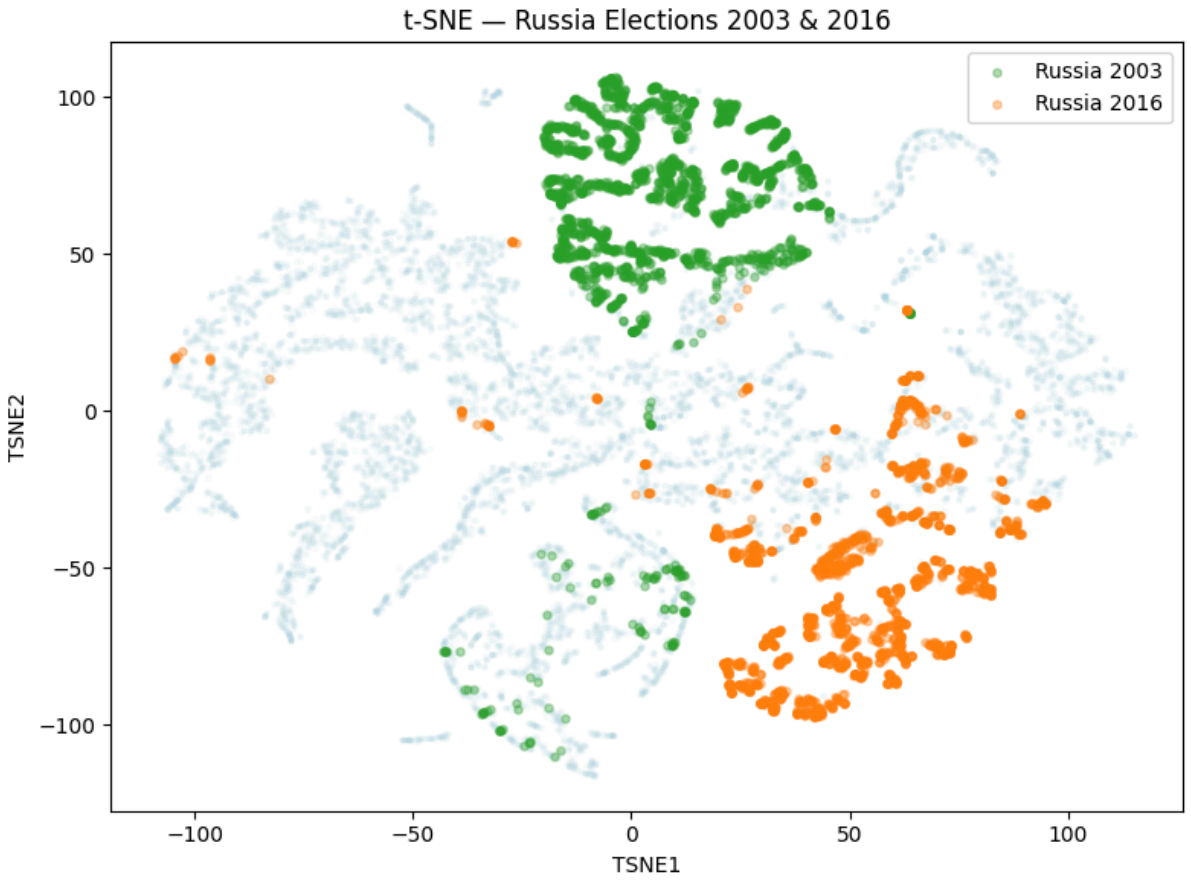

Fig 3.2.9 : t-SNE for Russian elections of 2013 and 2016

Here, the figure shows the result of the t-SNE on the modified subset, including 4,174 Russian Federation rows over the total of 20,000 rows. This modified subset helps us to see the behaviour of the specific country, but also includes the context of the other countries. Here we can see that both elections are unique and aren't really close to any other cluster of points. The two elections of 2003 and 2016 are clearly separated. This can indicate a reform in the electoral system, a shift in the voting behaviour of the citizens, or even a change in the way the results are counted by authorities.

We then looked to strengthen our results by performing our DBSCAN clustering method on data dimensionally reduced by the UMAP technique, saving more of the intercluster distance.

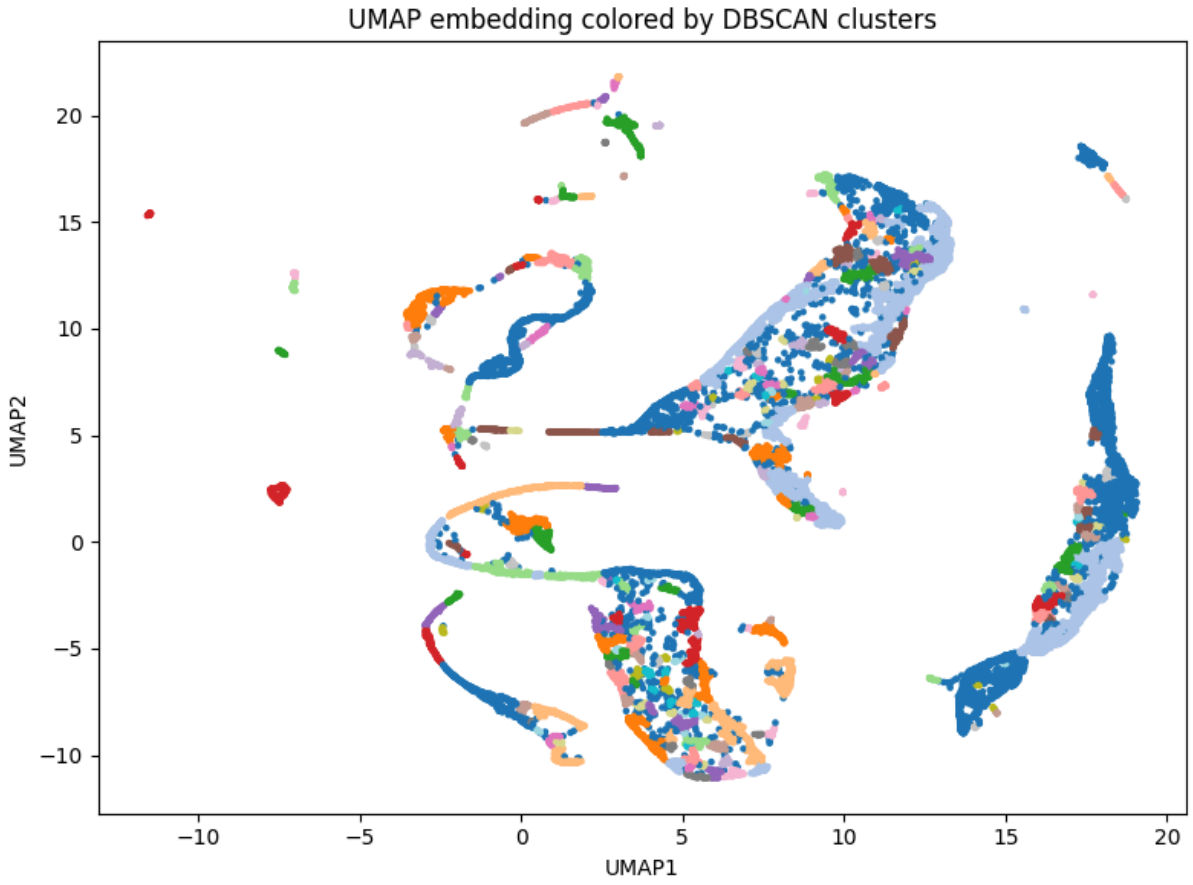

Fig 3.2.10 : UMAP embedding colored by DBSCAN clusters

This figure shows the DBSCAN clustering performed on the dimensionally reduced data by the UMAP technique. We can see that the clusters are more compact and that the DBSCAN is not that efficient. It is harder to interpret with the present clustering.

This is why we thought that it was more relevant to only use UMAP to look at the distance between the countries in this representation, and compare it to the equivalent figure using the t-SNE method. We want to be able to take into consideration the distance between the countries to conclude on the difference in behaviour between the two countries' elections, which can be performed using UMAP.

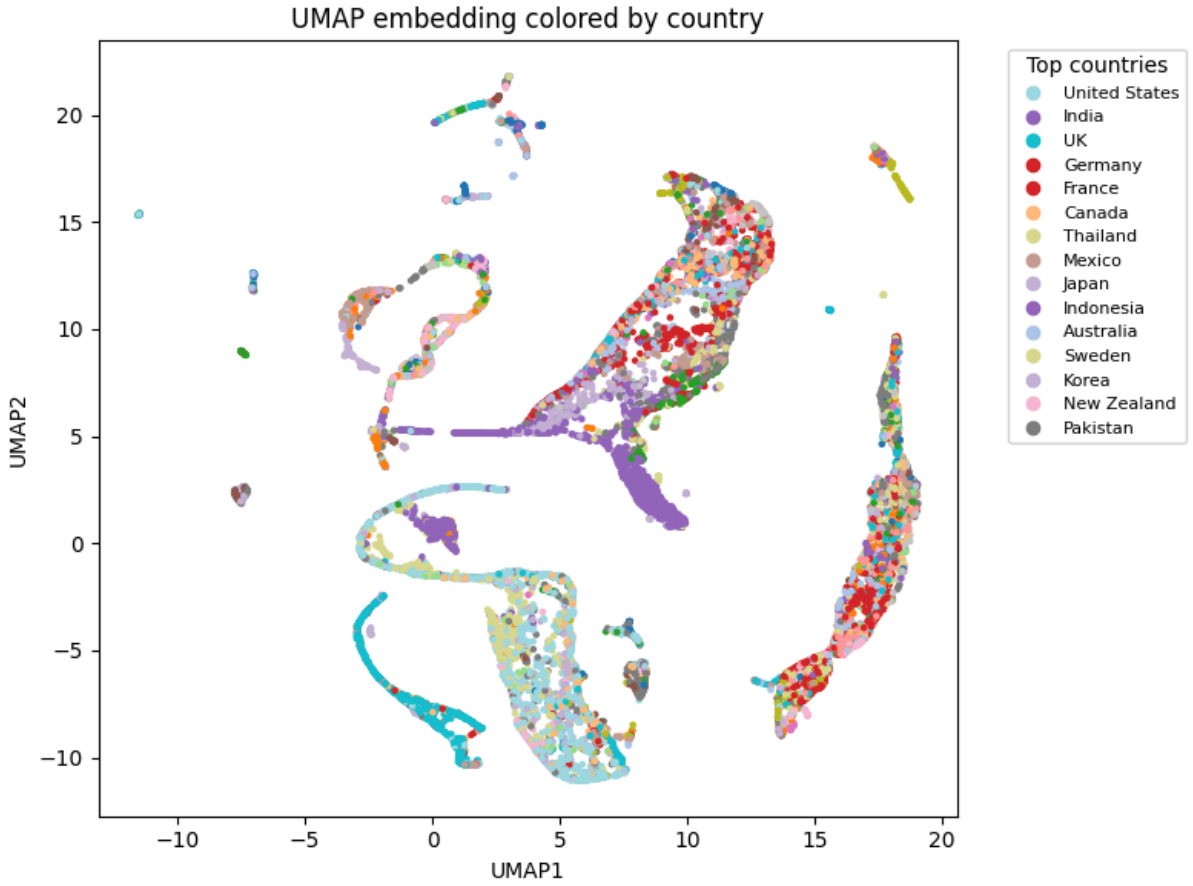

Fig 3.2.11 : UMAP embedding colored by country

This figure shows the UMAP embedding, where each point is colored by country. We can again identify some clusters mixed with multiple countries, while others, like for the UK, are only composed of the country's election. This UMAP allow us to confirm that this cluster is far away from the cluster including the Indian elections. We can also conclude that the electoral system of the UK is not really far away from the USA system. This result is logical regarding the historical link between the two countries, but also their actual links as Western democracies.

This last plot will be relevant to understanding the limits of our analysis. Considering a legislative election happens on average every 5 years, we estimated missing elections per country and plotted this histogram.

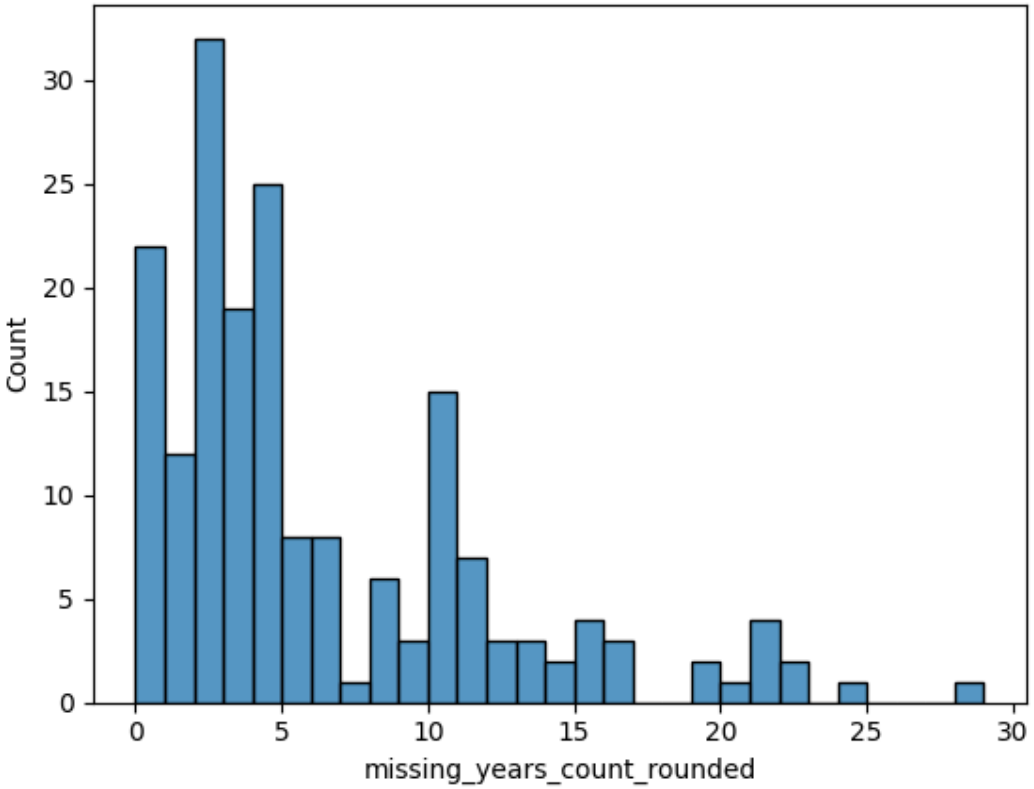

Fig 3.2.12 : Missing elections distribution

4. Discussion

4.1. Interpretation of Results

Figure 3.2.9 shows that the Russian Federation elections of 2003 and 2016 are organised with two different electoral systems in the dataset. Turchenko (2015) explains that a major reform was passed in Russia in 2014. This reform reintroduced the mixed-member majoritarian representation system, with an equal balance between the Single-Member Plurality and the Proportional Representation mandates. The 2003 election was also held under a mixed-member majoritarian representation system, but with different terms, such as a different proportion of 300-150 instead of 225-225. Despite electoral system reform, the electoral behaviour of Russian citizens also changed. Hale (2015) documents this change, describing a shift post-2012 Medvedev mandate toward a more personified support for Vladimir Putin. This change in the voting behaviour is captured in the dataset and can be one of the keys to understanding this two-cluster one-country phenomenon.

4.2. Limitations

Although we tried to bring meaningful insight into legislative elections categorization by their turnover and cast votes, we acknowledge limitations to our study.

First, the missing data are way bigger than just displayed in the method part. We can observe a great imbalance between represented countries. Morocco only has one election represented, in 2016, but Germany has data since the 19th century. Countries that existed and disappeared are missing, although they held legislative elections, such as Yugoslavia.

On average, countries hold elections on average every 5 years. Plot 3.2.11 shows us the distribution of elections unrepresented per country since the first election logged in the dataframe. A lot of countries have more than 1 century worth of elections missing. This clearly indicates that our study lacks consistent data.

Additionally, this study clearly highlights the difference between systems. A deeper study would compare political regimes that are alike, rather than comparing every election on a worldwide level.

Finally, we didn’t go deep enough in the qualitative study, as we kept it mainly quantitative. A future study could dive into qualitatively explaining differences between clusters, using proper comparative political science.

5. References

5.1 Datasets

Constituency-Level Lower Chamber Elections Archive | THEA. (October 15, 2025). https://electiondataarchive.org/data-and-documentation/clea-lower-chamber-elections-archive/

5.2 Books

Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (2nd ed.). O'Reilly Media.

Hale, H. E. (2015). *Patronal politics: Eurasian regime dynamics in comparative perspective*. Cambridge University Press.

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning: With applications in Python* (2nd ed.). Springer.

Lijphart, A. (2012). *Patterns of democracy: Government forms and performance in thirty-six countries* (2nd ed.). Yale University Press.

5.3 Papers

McInnes, L., Healy, J., & Melville, J. (2018). *UMAP: Uniform manifold approximation and projection for dimension reduction*.

Moses, L., & Box-Steffensmeier, J. M. (2020). *Considerations for machine learning use in political research with application to voter turnout*.

Turchenko, M. (2015). *Executive branch and major electoral reforms in Russia*. HSE / SSRN.

Van der Maaten, L. (2014). *Accelerating t-SNE using tree-based algorithms*. Journal of Machine Learning Research, 15, 3221–3245.

6. Appendices

6.1 Libraries

```
import numpy as np
import pandas as pd
import seaborn as sns
import missingno as msno
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
from matplotlib.patches import Patch
from matplotlib.lines import Line2D
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.cluster import KMeans, DBSCAN
from sklearn.neighbors import NearestNeighbors
from kneed import KneeLocator
import plotly.express as px
import hdbscan
```

Data Processing :

```
df_a_j = pd.read_excel("Data/clea_lc_20240419_excel/clea_lc_20240419_A-J.xlsx")
df_k_z = pd.read_excel("Data/clea_lc_20240419_excel/clea_lc_20240419_K-Z.xlsx")
df = pd.concat([df_a_j, df_k_z], ignore_index=True)
df1 = df[['pev1', 'vot1', 'vv1', 'ivv1', 'to1', 'cv1', 'cvs1', 'pv1', 'pvs1']]
df1.replace([-990, -994, -992], pd.NA, inplace=True)
df1.dropna(axis=0, inplace=True)
```

6.2 Custom Missing Matrix

```
def plot_custom_missing_matrix(
    df: pd.DataFrame,
    custom_na_values=(-990, -992, -994),
    show_true_nan=True,
    sort_by_completeness=True,
    figsize=(12, 6),
):
    """
    Visualizes a DataFrame like missingno.matrix but distinguishes multiple
    placeholder 'NA' codes by color. Optionally also shows real NaN as its own color.
```

```
Classes:
  0 -> present/valid values
  1..k -> each custom NA value in the order given
  (k+1) -> true NaN (if show_true_nan=True)
"""

# Build class matrix
n_rows, n_cols = df.shape
arr = np.zeros((n_rows, n_cols), dtype=np.int16)

# Mark each custom code with its own class id
for cls_idx, code in enumerate(custom_na_values, start=1):
    mask = df.eq(code).to_numpy()
    arr[mask] = cls_idx

# Optionally mark true NaN as a separate class
last_class = len(custom_na_values)
num_classes = last_class + 1  # +1 for "present"

# Create a simple qualitative colormap with distinct colors.
# First color = "present", next = one per custom NA, last = true NaN (if enabled).
cmap_colors = [
    "#e0e0e0",        # present (light gray)
    "#1f77b4",        # custom NA 1
    "#ff7f0e",        # custom NA 2
    "#2ca02c",        # custom NA 3
    "#d62728",        # true NaN (optional)
]
if num_classes > len(cmap_colors):
    # if there are more classes than colors above, extend with a color cycle
    cycle = ["#9467bd", "#8c564b", "#e377c2", "#7f7f7f", "#bcbd22", "#17becf"]
    need = num_classes - len(cmap_colors)
    cmap_colors.extend((cycle * ((need // len(cycle)) + 1))[:need])
cmap = ListedColormap(cmap_colors[:num_classes])

# Plot
fig, ax = plt.subplots(figsize=figsize)
im = ax.imshow(arr, aspect="auto", interpolation="nearest", cmap=cmap)
```

```
    # Ticks & labels similar to missingno
    ax.set_yticks([])
    ax.set_xticks(np.arange(n_cols))
    ax.set_xticklabels(df.columns, rotation=45, ha="right")
    ax.set_xlabel("Columns")
    ax.set_ylabel("Rows (sorted by completeness)" if sort_by_completeness else "Rows")
    # Column separators to echo missingno's style
    for x in range(-1, n_cols):
        ax.axvline(x + 0.5, linewidth=0.5, color="white", alpha=0.7)

    # Build legend
    legend_elems = [Patch(facecolor=cmap_colors[0], label="Present")]
    for i, code in enumerate(custom_na_values, start=1):
        legend_elems.append(Patch(facecolor=cmap_colors[i], label=f"{code}"))
    if show_true_nan:
            legend_elems.append(Patch(facecolor=cmap_colors[len(custom_na_values) + 1],
       label="NaN"))

    ax.legend(
        handles=legend_elems,
        title="Value",
        bbox_to_anchor=(1.02, 1), loc="upper left", borderaxespad=0.0
    )
    plt.tight_layout()
    plt.show()

plot_custom_missing_matrix(df, custom_na_values=[-990, -992, -994], show_true_nan=True)
```